Introduction to Programming and Data Structures

Data Structures - Queue

Malay Bhattacharyya

Associate Professor

MIU, CAIML, TIH
Indian Statistical Institute, Kolkata

October, 2023

1 Basics

2 Implementing a Queue
- Standard implementation
- Efficient implementation

3 Applications of Queue
- Breadth-first Search (BFS)

4 Problems



Traffic Management
Network protocols
Web servers
Task Scheduling
Resource Allocation

Motivating example



Problem: Maintain a list of patients waiting to consult a doctor.

Data structure: Queue (FIFO)

Operations:

- `INSERT` or `ENQUEUE`: insert at the end
- `DELETE` or `DEQUEUE`: remove and return element from the front
- `DISPLAY`: Show the elements
- `LENGTH` or `SIZE`
- `ISEMPTY`, `ISFULL`

Standard implementation – Initialization

Considering the queue size as static:

```
QueueDS = []
SizeQueueDS = 10
```

Standard implementation – Insert operation

```
def QueueINSERT(QueueDS, Data):
    if len(QueueDS) < SizeQueueDS:
        QueueDS.insert(0, Data)
    else:
        print("Queue overflow!!!")
```

Standard implementation – Delete operation

```
def QueueDELETE(QueueDS):
    if len(QueueDS) > 0:
        Data = QueueDS.pop()
        return Data
    else:
        print("Queue underflow!!!")
```

Note: If you do not wish to receive the deleted element, exclude the return statement.

Standard implementation – Displaying the elements

```
def QueueDISPLAY(QueueDS):
    print("The elements in the queue are: ")
    for Element in QueueDS:
        print(Element)
```

Note: You can directly write `print(QueusDS)`.

Drawbacks of Queue

1) not space efficient. (front=rear : queue empty , but size (free memory) exist)
In a linear queue, once elements are dequeued, the space they occupied cannot be reused unless the queue is emptied and all elements are shifted.

2) Complexity for Resizing: If you want to increase the size of a linear queue, you typically need to create a new, larger array, copy the existing elements to the new array, and update the pointers.

3) In a linear queue, when elements are dequeued, all remaining elements may need to be shifted to fill the gap left by the removed element.

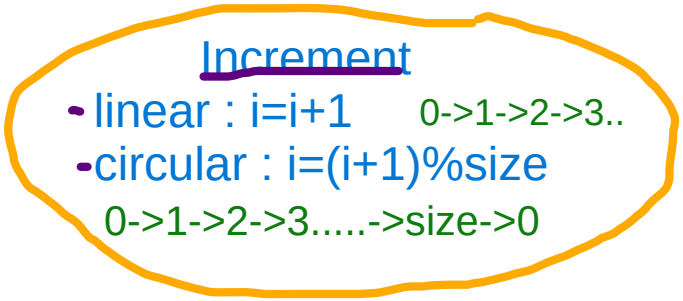
Efficient implementation – Initialization

CIRCULAR QUEUE

works like a regular queue, but with a circular arrangement of elements in memory. It has the advantage of efficiently utilizing available memory and providing constant time complexity for enqueuing and dequeuing operations

Considering the queue size as static:

```
QueueDS = []
SizeQueueDS = 10
```

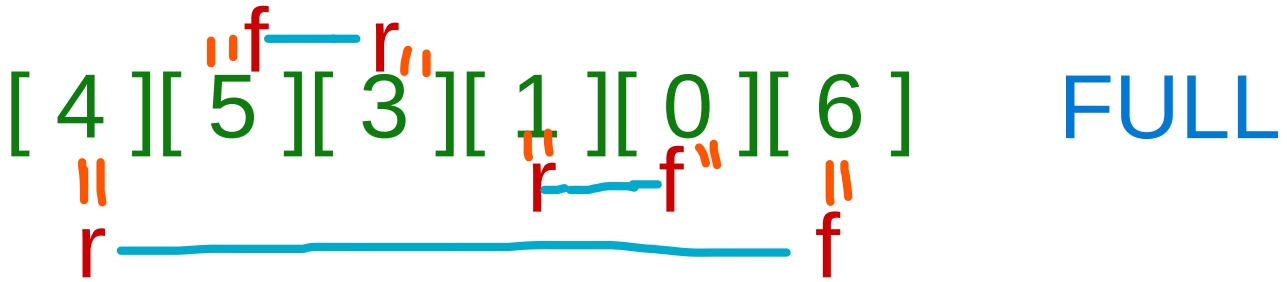


Initialization: A circular queue typically has an array of fixed size (capacity) and two pointers:

1) front : points to the front (or the first element) of the queue.
2) rear : points to the last element of the queue.
Initially, both front and rear are set to -1 to indicate that the queue is empty.

isEmpty() : when rear = front = -1
isFull() : when (rear+1)%size = front

Efficient implementation – Insert operation

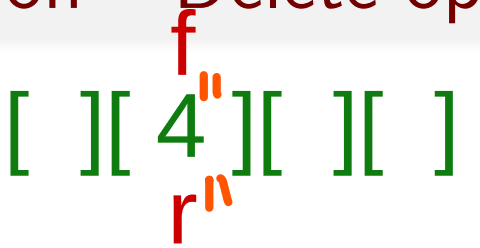



```
def QueueINSERT(QueueDS, Data):
    global rear
    if (rear + 1) % SizeQueueDS == front:
        print("Queue overflow!!!")
    else:
        QueueDS[rear] = Data
        rear = (rear + 1) % SizeQueueDS
```

Check if the queue is full: (rear + 1) % size == front. If true, the queue is full.

If not full, increment rear and add the element to the new rear position: rear = (rear + 1) % size

Efficient implementation – Delete operation




```
def QueueDELETE(QueueDS):
    global front, rear, SizeQueueDS
    if front == rear:
        print("Queue underflow!!!")
    else:
        Data = QueueDS[front]
        front = (front + 1) % SizeQueueDS
        return Data
```

Check if the queue is empty: front == -1. If true, the queue is empty.

If not empty, increment front to remove the element: front = (front + 1) % size.

Note: If you wish to receive the deleted element, include a return statement.

Efficient implementation – Displaying the elements




```
def QueueDISPLAY(QueueDS):
    print("The elements in the queue are: ")
    if front <= rear:
        print(QueueDS[front:rear])
    else:
        print(QueueDS[front:] + QueueDS[:rear])
```

T

to determine whether the queue has wrapped around (circularly) or not.

F

Note: You cannot directly write `print(QueusDS)`.



Breadth-first Search (BFS)

BFS explores all neighbors of a node before moving to their neighbors, making it suitable for finding the shortest path and exploring all vertices at a given depth level in a graph.

We can perform breadth-first search on a graph given in the form of adjacency list in a file. The BFS algorithm works as follows:

i) Insert any one of the graph's vertices in the front of a queue.

unweighted graph (all edges length same)

ii) Delete the rear data item from the queue and add it to the `Visited` list.

iii) Create a list of that vertex's adjacent nodes. Add the ones which are not in the `Visited` list to the front of the queue.

any order (thus not unique)

(breadth-first search)

iv) Repeat steps (ii)-(iii) until the queue is empty.

1) Initialization: Start from a source vertex. 2) Queue: Use a queue to maintain the order of exploration. 3) Visit: Mark the source vertex as visited. 4) Enqueue: Enqueue the source vertex into the queue. 5) Explore: While the queue is not empty:
Dequeue the front vertex from the queue.
Explore all unvisited neighbors of the dequeued vertex.
Mark each neighbor as visited and enqueue them.
6)Repeat: Continue this process until the queue is empty, ensuring that nodes are visited in a breadth-first manner

Implementing BFS

provides the shortest path to each vertex from the source

```
# Adjacency list defined as a dictionary
Graph = {
    '0' : ['1','2'], '1' : ['3', '4'], '2' : ['5'],
    '3' : [], '4' : ['5'], '5' : []
}
Visited = [] # Array to keep track of visited vertices
def BFS(Visited, Graph, Vertex):
    if Vertex not in Visited:
        Visited.insert(0, Vertex)
        print(Visited[0]) # Front of queue
        for Adjacent in Graph[Vertex]:
            BFS(Visited, Graph, Adjacent)
BFS(Visited, Graph, '0') # Function call with vertex '0'
```

Problems

1. Given an array and an integer S, find the minimum for each and every contiguous subarray of size S.
2. Given a list of numbers, find out the first non-repeating number in constant time (i.e., it should not depend on the total number of elements in the list).
3. Given a queue having multiple elements, write a recursive function to reverse its elements.